

AWS Lambda Runtime Migration Guide

Python 3.10 End-of-Support | October 2026

Enterprise Migration Playbook

About This Document

This playbook provides a structured, deployment-method-specific approach to migrating all AWS Lambda functions from Python 3.10 before the AWS end-of-support deadline in October 2026. Sections are organized by how each Lambda was originally deployed so teams can follow the path relevant to them.

1. Background & Why This Matters

AWS has announced end-of-support for the Python 3.10 Lambda runtime in October 2026. After this date, Lambda functions still running on 3.10 will no longer receive security patches, and AWS may block deployments or updates to those functions.

The upgrade target is Python 3.12 (recommended) or Python 3.13. Both runtimes include performance improvements, better typing support, and continued AWS managed patching.

Item	Detail
Deprecated Runtime	Python 3.10
End-of-Support Date	October 2026
Recommended Target	Python 3.12 (stable, widely tested)
Alternative Target	Python 3.13 (latest, fewer ecosystem libraries)
AWS Action After Deadline	Security updates cease; deployments may be blocked

2. Audit & Inventory

Before any migration work begins, you must have a complete picture of every Lambda in your environment. Skipping this step leads to missed functions and compliance gaps.

2.1 Multi-Account Discovery

Most enterprise AWS environments span multiple accounts and regions. Use one of the following approaches:

- AWS Config Aggregator enables cross-account, cross-region Lambda runtime reporting from a central account
- AWS Organizations + CLI scripting iterate over all accounts and regions using the Organizations API to get account IDs, then query Lambda per account
- AWS Resource Explorer newer service that indexes resources across accounts with tag and attribute filtering

2.2 CLI Inventory Script

The following command lists all Lambdas in a single account and region, filtering for Python 3.10:

```
# List all Python 3.10 Lambdas in current account/region
```

```
aws lambda list-functions \
  --query
'Functions[?Runtime==`python3.10`].[FunctionName,Runtime,LastModifiedDate]' \
  --output table

# Export to CSV for tracking
aws lambda list-functions \
  --query
'Functions[?Runtime==`python3.10`].[FunctionName,Runtime,LastModifiedDate]' \
  --output text | sed 's/\t/,/g' > lambda_audit.csv
```

3. Categorization & Risk Assessment

Not all Lambdas carry the same migration risk. Segment your inventory into the following buckets before assigning work to better understand how difficult it could be for the lambda to be compliant.

Category	Description	Migration Effort
Simple	No external dependencies, no layers, minimal code	Low runtime bump only
Dependency-heavy	Many pip packages in requirements.txt or Lambda layers	Medium dependency compatibility testing required
Business-critical	Customer-facing, revenue-impacting, or compliance-related	High full test cycle, change window needed
Orphaned / Unknown	No clear owner, low or zero recent invocations	Assess for decommission before migrating

✓ **Tip:** Use CloudWatch Invocation metrics over the past 90 days to identify orphaned functions. If a function has had zero invocations in 90 days and has no clear owner, it is a candidate for decommission rather than migration reducing scope saves time.

4. Migration Approach by Deployment Method

How Lambda was originally deployed determines how it should be migrated. The following sections cover each deployment method in detail.

4.1 Infrastructure-as-Code: Terraform

Terraform-managed Lambdas are the most straightforward to migrate at scale because the runtime is declared explicitly in code and changes can be reviewed via plan before being applied.

Step-by-Step

1. Find all Lambda resource blocks using runtime = 'python3.10' across your Terraform codebase
2. Update the runtime argument to python3.12 (or python3.13)
3. Update any Lambda layers that were compiled for 3.10 these must be rebuilt for the target runtime
4. Update requirements.txt and re-run pip install to validate dependency compatibility
5. Run terraform plan to review the changeset confirm only runtime (and any layer ARN) changes, not resource recreation
6. Apply to dev/staging first and run integration tests
7. Apply to production within an approved change window

```
# Before
resource "aws_lambda_function" "my_function" {
  function_name = "my-function"
  runtime       = "python3.10"    # <-- update this
  handler       = "handler.main"
  ....
}

# After
resource "aws_lambda_function" "my_function" {
  function_name = "my-function"
  runtime       = "python3.12"
  handler       = "handler.main"
  ....
}
```

⚠ Warning: terraform plan may show resource recreation if the function has a filename or source_code_hash change alongside the runtime update. Always review the full plan output carefully before applying to production.

4.2 Infrastructure-as-Code: AWS CDK

CDK Lambdas declare the runtime via the `lambda.Runtime`. The change is a one-liner in most cases, but CDK synthesizes CloudFormation under the hood, so the same discipline around reviewing the changeset applies.

Step-by-Step

8. Search your CDK app for `lambda.Runtime.PYTHON_3_10` (TypeScript/JavaScript) or `_lambda.Runtime.PYTHON_3_10` (Python CDK)
9. Replace with `lambda.Runtime.PYTHON_3_12`
10. Rebuild any Lambda layers use the `BundlingOptions` or custom build scripts to target the new runtime
11. Run `cdk diff` to review the synthesized CloudFormation changeset
12. Deploy to dev, test, then promote to production

```
// TypeScript Before
new lambda.Function(this, 'MyFunction', {
  runtime: lambda.Runtime.PYTHON_3_10, // <-- update
  handler: 'handler.main',
  code: lambda.Code.fromAsset('src'),
});

// TypeScript After
new lambda.Function(this, 'MyFunction', {
  runtime: lambda.Runtime.PYTHON_3_12,
  handler: 'handler.main',
  code: lambda.Code.fromAsset('src'),
});
```

✓ **Tip:** If you use CDK Pipelines, update the runtime in the shared stack construct or base class so all downstream Lambdas inherit the change automatically this can eliminate dozens of individual changes in one commit.

4.3 Infrastructure-as-Code: AWS SAM

SAM templates (template.yaml) declare runtime as a top-level property on each `AWS::Serverless::Function` resource, or as a global default under `Globals`.

Step-by-Step

13. Open your template.yaml and search for `Runtime: python3.10`
14. If set under `Globals: Function:`, update it once all functions inherit it unless overridden
15. If set per-function, update each one individually
16. Run `sam build` to rebuild the deployment artifact against the new runtime
17. Run `sam local invoke` to test locally before deploying
18. Deploy with `sam deploy` and validate

```
# template.yaml Global approach (preferred for consistency)
Globals:
  Function:
    Runtime: python3.12    # was python3.10
    Timeout: 30

Resources:
  MyFunction:
    Type: AWS::Serverless::Function
    Properties:
      Handler: handler.main
      # Runtime inherited from Globals
```

⚠ Warning: If individual functions override the global Runtime with python3.10, those overrides will not be caught by updating Globals alone. Always grep your full template for python3.10 after making the global change.

4.4 AWS Console-Deployed Lambdas

Lambdas created directly in the AWS Console are the most operationally risky to migrate because there is no IaC state to diff, no code repository to trace, and often no clear ownership. Treat console-deployed Lambdas as a priority audit item.

Identifying Console-Deployed Lambdas

Console-created Lambdas often share these characteristics:

- No corresponding Terraform state, CDK stack, or SAM template references the function ARN
- Created or last modified by an IAM user (not a CI/CD role)
- Function code is inline (editor-based) rather than from an S3 artifact or ECR image
- No deployment pipeline invocations in CloudTrail for the function

Migration Approach

● **Important:** Before migrating to a console Lambda, always export the function code and configuration first. Console Lambdas have no version-controlled source of truth if something goes wrong, you need a backup.

19. Export the function: download the deployment package via Console or CLI
20. Document the function configuration: environment variables, layers, VPC config, timeout, memory, triggers
21. Introducing the function into IaC (Terraform, CDK, or SAM) this is the ideal opportunity to bring it under version control
22. Test the new IaC-managed version in dev/staging before decommissioning the console version
23. If IaC adoption is not feasible in the timeline, update the runtime directly via Console or CLI:

```
# Update runtime directly via CLI (console-deployed Lambdas)
aws lambda update-function-configuration \
  --function-name my-console-function \
  --runtime python3.12

# Verify the update
aws lambda get-function-configuration \
  --function-name my-console-function \
  --query '[FunctionName, Runtime]'
```

✓ **Tip:** Use this migration as the forcing function to bring console Lambdas into IaC. A Lambda that was created ad-hoc in the Console will create the same problem at the next runtime deprecation if not brought under version control now.

4.5 CI/CD Pipeline-Deployed Lambdas

Lambdas deployed through a CI/CD pipeline (GitHub Actions, GitLab CI, Jenkins, AWS CodePipeline, etc.) have the runtime defined somewhere in their pipeline configuration or build scripts. The approach depends on where the runtime is declared.

Common Runtime Declaration Locations

- In a buildspec.yml or pipeline YAML alongside the SAM/Terraform command
- In a Makefile or shell deploy script passed as an environment variable
- In a pipeline-level variable or secret store that populates the IaC at build time

Step-by-Step

24. Trace the pipeline to find where python3.10 is declared or passed
25. Update the value in the pipeline definition, variable group, or environment variable
26. If the runtime is hardcoded in a build script, update the script and commit
27. Trigger a pipeline run against a dev environment to validate
28. Promote through staging to production via the normal pipeline promotion process
29. Enforce the new runtime via a pipeline policy check fail builds that attempt to deploy python3.10

```
# Example: GitHub Actions workflow snippet
- name: Deploy Lambda
  run: |
    sam deploy \
      --parameter-overrides \
        Runtime=python3.12 \      # was python3.10
      --stack-name my-stack \
      --capabilities CAPABILITY_IAM
```

4.6 Container Image-Based Lambdas

Lambdas deployed as container images do not use the runtime field the Python version is instead determined by the base image in the Dockerfile. These are migrated differently.

Step-by-Step

30. Open the Dockerfile for the Lambda and locate the FROM instruction
31. Update the base image from the 3.10 AWS Lambda base image to 3.12
32. Rebuild the container image locally and run unit tests
33. Push the new image to ECR and update the Lambda to point to the new image URI
34. Monitor cold start times and memory usage Python 3.12 images may differ in size

```
# Dockerfile Before
FROM public.ecr.aws/lambda/python:3.10
```



```
# Dockerfile After
FROM public.ecr.aws/lambda/python:3.12

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY handler.py .
CMD ["handler.main"]
```

⚠ Warning: Any pip packages that were pre-compiled as C extensions (numpy, cryptography, etc.) inside the 3.10 image must be recompiled against 3.12. Do not reuse compiled artifacts from one Python version inside an image built on another.

5. Common Pitfalls & Compatibility Issues

Python 3.12 introduced several changes that can cause silent runtime failures if not accounted for:

Pitfall	Description	Mitigation
distutils removed	distutils was deprecated in 3.10 and fully removed in 3.12	Replace with setuptools; update any package that depends on distutils internally
boto3 version mismatch	The bundled boto3 in the Lambda runtime may differ from the version assumed in 3.10	Pin boto3 and botocore explicitly in requirements.txt
Lambda Layers compiled for 3.10	Binary layers compiled for one Python version are incompatible with another	Rebuild all layers targeting python3.12
Implicit string encoding changes	Some encoding behaviors changed between 3.10 and 3.12	Run the test suite; pay attention to any bytes/str handling
Third-party library support	Some older libraries have not published 3.12-compatible wheels	Check PyPI compatibility; find alternatives or update library versions
Type annotation changes	3.12 enforces some typing rules more strictly	Review functions that use complex generics or older annotation patterns

6. Testing Strategy

All migrated Lambdas should pass through the following testing gates before promotion to production:

6.1 Local Testing

- Update local Python version to 3.12 (use pyenv or a virtual environment)
- Run `pip install -r requirements.txt` and resolve any dependency errors before touching AWS
- Execute the unit test suite and confirm all tests pass
- For SAM functions, use `sam local invoke` to simulate the Lambda runtime locally

6.2 Dev / Staging Validation

- Deploy to a non-production environment first never migrate directly to production
- Send synthetic or replayed production traffic if possible
- Monitor CloudWatch Logs for errors in the first 24 hours post-deployment

- Confirm that CloudWatch Metrics (Errors, Duration, Throttles) are within baseline

6.3 Production Promotion

- Use Lambda Aliases and Weighted Routing to shift 10% of traffic to the new runtime first
- Monitor for 30 minutes before completing the cutover
- Retain the previous function version for at least 72 hours to enable rollback
- Update the migration-status tag to be done and record the completion date

7. Preventing Future Drift

Once migration is complete, put guardrails in place so Python 3.10 (or any newly deprecated runtime) can never re-enter the environment:

- AWS Config Rule creates a custom or managed rule that flags any Lambda with a deprecated runtime and alerts the security/platform team
- CI/CD gate add a pre-deployment check in all pipelines that fail the build if the runtime is not on the approved list
- Service Control Policy (SCP) in multi-account environments, use SCPs to deny creation or update of Lambda functions with deprecated runtimes
- IaC linting integrates tools like tfint (Terraform) or cdk-nag (CDK) that can enforce runtime policies as part of the developer inner loop

8. Recommended Timeline

The following phased timeline is designed to complete all migrations at least four weeks before the October 2026 AWS deadline:

Phase	Activity	Target Window
1 Audit	Complete Lambda inventory across all accounts and regions; apply ownership tags; identify console vs IaC vs pipeline deployed	Now > End of May 2026
2 Quick Wins	Migrate all low-risk, dependency-light Lambdas; update runtimes in IaC with simple find-and-replace; deploy through CI/CD	June 2026
3 Complex	Migrate dependency-heavy and container image Lambdas; rebuild layers; coordinate with owning teams on testing	July 2026
4 Critical	Migrate business-critical Lambdas; use weighted routing; extended monitoring windows; full rollback plans in place	August 2026

Phase	Activity	Target Window
5 Console	Migrate or decommission all remaining console-deployed Lambdas; bring survivors into IaC	August early September 2026
6 Buffer	Final audit sweep; resolve any stragglers; validate Config rule compliance is 100%; enforcement SCPs active	September 2026
DEADLINE	All Lambdas on Python 3.12 or 3.13. AWS end-of-support takes effect.	October 2026

9. Team Ownership Model

On a scale, the migration cannot be owned by a single central team. The recommended operating model is:

Central Platform Team Responsibilities

- Maintain the master inventory and migration tracking dashboard (AWS Config Aggregator or a shared spreadsheet/Jira board)
- Publish and own the migration playbook (this document)
- Provide shared tooling: CLI scripts, pipeline policy checks, Config rules
- Escalation points for teams encountering compatibility blockers
- Report progress to leadership weekly

Owning Team Responsibilities

- Migrate all Lambdas tagged to their team by the agreed deadline
- Follow the playbook for their deployment method
- Update the migration-status tag as work progresses
- Raise blockers to the platform team with at least three weeks of buffer before the deadline

✓ **Tip:** Send a monthly migration scorecard to all engineering leads showing their team's completion percentage. Social visibility, not just compliance, is one of the most effective drivers of on-time delivery for org-wide migrations.

10. Quick Reference Checklist

Use the following checklist for each Lambda function during migration:

#	Checklist Item	Done?
1	Function identified in inventory and tagged with owner and criticality	☐
2	Deployment method confirmed (Terraform / CDK / SAM / Console / Pipeline / Container)	☐
3	Runtime updated in source (IaC, Dockerfile, or CLI)	☐
4	Lambda layers rebuilt for Python 3.12	☐
5	requirements.txt updated and tested locally	☐
6	Unit tests passing on Python 3.12	☐
7	Deployed and validated in dev / staging	☐
8	Production deployment completed with weighted routing	☐
9	CloudWatch metrics is nominal for 24 hours post-migration	☐
10	migration-status tag updated to 'done'	☐
11	Previous function version retained for rollback (72 hours minimum)	☐